

Intelligent Candidate Discovery & Ranking

Redrob Hackathon · Team akulasanith18 Chunchu Venugopal & Sanith Akula

A multi-signal hybrid scoring system that reads what candidates actually did — not just what they listed.

Runtime	Memory	GPU	LLM API calls
~90 sec	<2 GB RAM	None	None

1. The Problem — Why Keyword Matching Fails

The JD explicitly warns: the right answer is NOT 'find candidates whose skills section contains the most AI keywords.' Two failure modes to avoid:

Trap	Example	Correct handling
Keyword stuffer	Marketing Manager who listed "RAG" and "Direct" as skills	Career history reveals mismatch
Plain-language fit	Built recommendation system at product co	Career analysis finds "RAG" anyway
Ghost candidate	Perfect profile, inactive 8 months, 5% response rate	Behavioral signals down-weight them
Honeypot profile	"Expert" in 10 skills, 0 months usage; 8yr exp at S&P	Explicit rejection → score 0
Pure services	Entire career at TCS/Wipro/Infosys — JD says "bad fit"	Bad fit

2. Architecture — Multi-Signal Hybrid Scorer

Score = weighted sum of 5 independent signal groups. Max 100 points, normalized to 0–1.

Component	Max pts	Key signals used
Core skill match	35	Proficiency × duration × JD-critical skill set + career text NLP
Career quality	30	YoE 5-9yr sweet spot, product vs services ratio, shipped retrieval systems,
Behavioral signals	20	Recency, open-to-work, notice period, response rate, interview completion
GitHub activity	8	Commit/PR score — external validation of open-source engagement
Location fit	7	India cities (Pune/Noida/Hyderabad/Bangalore/Mumbai/Delhi) or willing to r

Why this over embeddings?

Embedding cosine similarity over skill names misses semantic intent. A candidate who 'built a vector search index serving 10M queries/day at a fintech startup' may not use the exact term 'dense retrieval' — but our career text scan catches them. Conversely, a candidate who lists every buzzword as a skill but whose titles are 'Delivery Manager' throughout gets filtered by career quality scoring.

3. Skill Scoring — Beyond Tag Matching

Two-layer skill analysis:

- **Layer 1 — Structured skill tags** (proficiency × usage duration): Each skill in `candidate.skills[]` scored as `proficiency_weight × (0.6 + 0.4 × min(duration_months/60, 1))`. Proficiency: expert=1.0, advanced=0.8, intermediate=0.5, beginner=0.2.
- **Layer 2 — Career text NLP**: Scan all career description narratives + summary for JD-critical keywords. Catches candidates who used different terminology for the same concepts (e.g. 'built recommendation engine' vs 'retrieval system').

JD-critical skill clusters targeted:

Cluster	Skills included
Embeddings & retrieval	sentence-transformers, OpenAI embeddings, BGE, E5, dense retrieval, semantic search
Vector infrastructure	Pinecone, Weaviate, Qdrant, FAISS, Milvus, Elasticsearch, OpenSearch, hybrid search
Ranking systems	BM25, NDCG, MRR, learning-to-rank, re-ranking, A/B testing, evaluation frameworks
LLMs & fine-tuning	LLM, RAG, fine-tuning, LoRA, QLoRA, PEFT, Hugging Face
Core ML	PyTorch, scikit-learn, XGBoost, NLP, transformers, BERT, GPT

4. Career Quality — Reading Between the Lines

What the JD actually means (vs what it says)

- **Years of experience 5–9** is a signal of judgment, not a hard filter. Scored as: +8 pts for 5–9yr, +5 pts for 4–5yr or 9–12yr, +2 pts for 3–4yr, 0 otherwise.
- **Product company ratio**: Career months at product vs total career months. >60% product = +10 pts. >30% mixed = +6 pts. >80% services = –5 pts (JD says 'bad fit twice').
- **Shipped retrieval/ranking/search**: Career descriptions scanned for: 'ranking', 'retrieval', 'recommendation', 'search', 'vector', 'embedding', 'matching'. 3+ mentions = +8 pts (evidence of actually shipping, not just knowing).
- **Production scale evidence**: 'production', 'deployed', 'real users', 'scale', 'latency', 'serving', 'million' in narratives = +4 pts.
- **CV/Speech domain penalty**: If candidate has >3 CV/speech skills AND no NLP/IR core skills → –4 pts (JD explicitly says 'you'd be relearning fundamentals').

Companies classified as pure IT services (penalty if >80% career):

TCS · Infosys · Wipro · Accenture · Cognizant · Capgemini · HCL · Tech Mahindra · Mphasis · Hexaware · Mindtree · L&T Infotech · IBM · Deloitte · PwC · KPMG

5. Behavioral Signals — Is the Candidate Actually Hirable?

A perfect-on-paper candidate who hasn't logged in for 6 months and has a 5% response rate is, for hiring purposes, not actually available. We use all 23 Redrob signals, weighted by hiring relevance:

Signal	Scoring logic	Pts
last_active_date	≤30d: +5 ≤90d: +3 ≤180d: +1 >180d: -2 (ghost)	±5
open_to_work_flag	True: +3 (explicit availability)	+3
notice_period_days	≤30d: +4 ≤60d: +2 >90d: -1 (JD wants sub-30d)	±4
recruiter_response_rate	≥70%: +4 ≥50%: +2 <15%: -2 (unresponsive)	±4
interview_completion_rate	≥80%: +2 <40%: -1 (reliability signal)	±2
verified_email + phone	Both verified: +1 (identity confidence)	+1
saved_by_recruiters_30d	≥5: +1 (social proof from other recruiters)	+1
github_activity_score	≥70: +8 ≥40: +5 ≥10: +2 -1: neutral	+8

6. Honeypot Detection & Compute Profile

Honeypot detection (score forced to 0.0):

- Duration claimed at a job > months since that company was started (timeline impossibility)
- Expert proficiency on 8+ skills with 0 months usage each (impossible expertise without practice)
- YoE claimed > 2× actual career history months, combined with many expert skills

Detected 20 honeypots in the 100K pool (challenge expected ~80 — our threshold is conservative to avoid false positives).

Compute profile:

Metric	Value
Runtime (100K candidates)	~90 seconds
Peak memory	<500 MB
External API calls	0
GPU requirement	None
Pre-computation required	None
Dependencies	Python stdlib only (json, csv, datetime, re, argparse)
Single command to reproduce	<code>python rank.py --candidates candidates.jsonl --out submission.csv</code>

7. Top-10 Ranked Candidates

These candidates were selected as the strongest fits for the Senior AI Engineer role based on our multi-signal scorer:

Rank	Candidate ID	Score	Key highlights
1	CAND_0018499	0.980	Sr ML Eng, 7.2yr, 11 core skills incl. qlora/bm25/embeddings, product co., India
2	CAND_0002025	0.960	Sr AI Eng, 5.9yr, 11 core skills, shipped ranking systems, mixed product bg
3	CAND_0011687	0.960	Sr NLP Eng, 7.8yr, pytorch/peft/embeddings/semantic search, India-based
4	CAND_0071974	0.960	Sr AI Eng, 7.8yr, 13 core skills incl. sentence-transformers, product co.
5	CAND_0046064	0.950	Sr NLP Eng, 8.9yr, python/qlora/peft/bm25/elasticsearch, strong retrieval bg
6	CAND_0052682	0.931	NLP Engineer, 6.6yr, 7 core skills, solid availability signals
7	CAND_0052328	0.930	Recommendation Systems Eng, 6.5yr, shipped recsys at product co.
8	CAND_0077337	0.930	Staff ML Eng, 7.0yr, broad skill coverage, India, open to work
9	CAND_0079387	0.930	AI Engineer, 6.9yr, sentence-transformers/bm25/semantic search focus
10	CAND_0041669	0.920	Recsys Eng, 8.0yr, qlora/peft/machine learning, strong career narrative

What these candidates have in common:

- 5–9 years of experience in applied ML/AI (the JD sweet spot)
- At least one product company in their career history
- Career narratives describing shipped retrieval, ranking, or recommendation systems
- Multiple core skills: sentence-transformers, FAISS/Elasticsearch, BM25, LoRA/QLoRA, Python
- India-based with manageable notice periods and high recruiter response rates

8. What We Built — Repository Overview

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

File	Purpose
rank.py	Main ranking script — single entry point, no pre-computation needed
submission.csv	Top-100 ranked candidates with scores and reasoning
submission_metadata.yaml	Portal metadata (team, approach summary, declarations)
requirements.txt	numpy, pandas (only standard libs actually used at runtime)
README.md	Setup, reproduce command, architecture overview

Design principles:

- **JD-first:** Every scoring rule traces directly to a specific statement in the job description. No generic recruiting heuristics.
- **Transparency:** Each candidate gets a human-readable reasoning string explaining why they ranked where they did.
- **Production-ready:** Streams through 100K candidates in a single pass. No RAM spike. No temp files.
- **Reproducible:** Pure Python, no model weights, no API keys. Anyone can run it.

Build something real. Make hiring smarter.

Team akulasanith18 · Chunchu Venugopal & Sanith Akula

The problem	The solution
Keyword stuffers rank high on naive systems	Career text NLP + title validation
Plain-language fits get buried	Career narrative scan finds them
Ghost candidates waste recruiter time	Behavioral signal down-weighting
Honeypots fool pure embedding systems	Impossible profile detection
Services-heavy CVs in tech roles	Company type ratio scoring